

Code 4.1 Program convert infix to postfix by assigned in variable.

ให้นักศึกษานำโค้ดโปรแกรมตามตัวอย่างในเอกสาร ไปทดสอบ ศึกษาการทำงาน และผลลัพธ์

```
#include <stdio.h>           //use printf()
#include <conio.h>            //use getch()
#include <string.h>           //use string function
#define MaxStack 40          // Set Max Operator Stack
char infix1[80] = {"A+B*(C^D*E/F)-G"}; // Assign INFIX
char OpSt[MaxStack];         // Operator stack size
int SP = 0;                  // Initial SP=0
```

เป็นการเรียกใช้ Library ที่จำเป็นต่อการทำงานของโปรแกรม

Stdio.h ใช้ฟังก์ชัน printf() สำหรับแสดงผลข้อมูลทางหน้าจอ

Conio.h ใช้ฟังก์ชัน getch() สำหรับหยุดหน้าจอรอการกดปุ่ม

String.h ใช้ฟังก์ชันเกี่ยวกับสตริง เช่น strlen() และ strchr()

#define MaxStack 40 กำหนดขนาดสูงสุดของ Stack เท่ากับ 40 ช่อง

Infix1 เก็บนิพจน์แบบ Infix ที่ต้องการแปลง

OpSt Operator Stack ใช้เก็บเครื่องหมาย Operator ช่วงการระหว่างการแปลง

Stack Pointer ใช้ชี้ตำแหน่งบนสุดของ Stack เริ่มต้นเป็น 0 หมายถึง Stack ว่าง

```
void push(char oper)          // PUSH Function
{
    if (SP == MaxStack) // Check Stack FULL?
    {
        printf("ERROR STACK OVER FLOW!!!...\n");
    }
    else
    {
        SP = SP + 1;      // Increase SP
        OpSt[SP] = oper;  // Put data into Stack
    }
}
```

ฟังก์ชัน Push ใช้สำหรับนำข้อมูลเข้า Stack ขั้นตอนการทำงาน ตรวจสอบว่า Stack เต็มหรือไม่

```

int pop() // POP Function
{
    char oper;
    if (SP != 0) // Check Stack NOT EMPTY?
    {
        oper = OpSt[SP]; // Get data from Stack
        SP--;           // Decrease SP
        return (oper);   // Return data
    }
    else
        printf("\nERROR STACK UNDER FLOW!!!...\n");
}

```

ฟังก์ชัน Pop ใช้สำหรับดึงข้อมูลออกจาก Stack

```

int precedenceIP(char oper) // Function for check precedence of inputoperator
{
    switch (oper)
    {
        case '+':
            return (1);
        case '-':
            return (1);
        case '*':
            return (2);
        case '/':
            return (2);
        case '^':
            return (4);
        case '(':
            return (4);
    }
}

```

ใช้ตรวจสอบลำดับความสำคัญของ Operator ที่รับเข้ามาใหม่

```

void infixToPostfix(char infix2[80])
{
    int i, j, len;
    char ch, temp;
    printf("INFIX : %s\n ", infix2);    // Show infix
    len = strlen(infix2);               // Find length of infix
    printf("Infix Length = %d \n", len); // Display length of infix
    printf("POSTFIX IS : ");
    for (i = 0; i <= len - 1; i++) // split infix
    {
        ch = infix2[i];               // Transfer character in to ch variable
        if (strchr("+-* /^()", ch) == 0) // Check Is OPERAND?
            printf("%c", ch);          // Out to Postfix
        else                           // If OPERATOR do below
        {
            if (SP == 0)                // Stack empty?
                push(ch);              // Push any way if Stack empty
            else if (ch != '(') // If not '(' do below
            {
                if (precedenceIP(ch) > precedenceST(OpSt[SP])) // If precedence input > precedence in stack
                    push(ch); // Push input operator to Stack
                else
                {
                    if (SP == 0) // Stack empty?
                        push(ch); // Push any way if Stack empty
                    else if (ch != '(') // If not '(' do below
                    {
                        if (precedenceIP(ch) > precedenceST(OpSt[SP])) // If precedence input > precedence in stack
                            push(ch); // Push input operator to Stack
                        else
                        {
                            printf("%c", pop()); // Out to Postfix
                            while (precedenceIP(ch) <= precedenceST(OpSt[SP]) && (SP != 0)) // Do Until precedence input > precedence in stack
                                printf("%c", pop()); // Out to Postfix
                            push(ch); // Push operator input to Stack
                        }
                    }
                }
            }
        }
        else
        {
            temp = pop(); // Pop operator from Stack
            while ((temp != '(')) // Do if not found '('
            {
                printf("%c", temp); // Out to Postfix
                temp = pop(); // Pop again and loop
            }
        }
    }
    j = SP; // Let j for count left Stack
    for (i = 1; i <= j; i++) // POP all if Input is NULL
        printf("%c", pop()); // Out to Postfix
}

```

แสดง Infix printf("INFIX : %s\n ", infix2); ผลลัพธ์ INFIX : A+B*(C^D*E/F)-G

หาความยาวข้อความ len = strlen(infix2); ผลลัพธ์ Infix Length = 15

วนลูปอ่านทีละตัวอักษร for (i = 0; i <= len - 1; i++) อ่านนิพจน์ทีละตัว ดังนี้

$A + B * (C ^ D * E / F) - G$

จากนั้นเข้าฟังก์ชัน ตรวจสอบว่าเป็น Operand หรือ Operator

ถ้าไม่พบเครื่องหมาย + - * / () แสดงว่าเป็น Operand

ให้นำออกสู่ Postfix ทันที printf("%c", ch); ตัวอย่าง Input A+B อ่าน A ผลลัพธ์ A

กรณีเป็น Operator ถ้า Stack ว่าง Push ลง Stack ได้ทันที

ถ้า Operator ใหม่มี Priority สูงกว่า ให้ Push ได้เลย

ถ้า Operator ใหม่มี Priority ต่ำกว่า printf("%c", pop()); และ Pop ต่อไปเรื่อย ๆ จนกว่าจะเจอ Operator ที่มี Priority ต่ำกว่า

กรณีเจอวงเล็บปิด) ให้ Pop ข้อมูลออกจาก Stack ให้ Pop Operator เรียงจาก Stack ได้เลยและลบ)

ถึงค่าที่เหลือใน Stack เมื่ออ่าน Infix ครบแล้ว ยังมี Operator ค้างใน Stack เช่น + - จะถูก Pop ออกมาจนหมด

```
int main()
{
    printf("INFIX to POSTFIX CONVERSION PROGRAM\n");
    printf("=====\n");
    infixToPostfix(infix1);
    getch();
    return (0);
} // End MAIN
```

เป็นฟังก์ชันหลักของโปรแกรม แสดงชื่อโปรแกรม เรียกฟังก์ชันแปลง Infix เป็น Postfix รอผู้ใช้กดปุ่มเพื่อจบการทำงาน

ผลลัพธ์โปรแกรม

```
● PS D:\Bachelor's\ENGCE124\lab4> ./infix-to-postfix
INFIX to POSTFIX CONVERSION PROGRAM
=====
INFIX : A+B*(C^D*E/F)-G
Infix Length = 15
POSTFIX IS : ABCD^E*F/*+G-
○ PS D:\Bachelor's\ENGCE124\lab4> █
```

Code 4.2 Program calculate Postfix by assigned in variable.

ให้นักศึกษานำโค้ดโปรแกรมตามตัวอย่างในเอกสาร ไปทดสอบ ศึกษาการทำงาน และผลลัพธ์

```
#include <stdio.h>           //use printf()
#include <conio.h>            //use getch()
#include <string.h>          //use string function
#include <math.h>            //use power
#define MaxStack 40         // Set Max Operator Stack
char postfix1[80] = {"AB+C-d/"}; // Assign INFIX
float ValPostfix[80];       // Keep value of Postfix here
float ValOperandST[MaxStack]; // Operator stack size
int SP = 0;                // Initial SP=0
```

ประกาศ Library ที่ใช้งานและกำหนดขนาด Stack สูงสุด 40 ช่อง

postfix1 เก็บนิพจน์ Postfix

float ValPostfix[80]; ใช้เก็บค่าตัวเลขของ Operand ค่าจะถูกเก็บไว้ใน Array

float ValOperandST[MaxStack]; เป็น Stack สำหรับเก็บตัวเลขระหว่างการคำนวณ

int SP = 0; Stack Pointer ใช้ชี้ตำแหน่งบนสุดของ Stack

```
void push(float ValOperand) // PUSH Function
{
    if (SP == MaxStack) // Check Stack FULL?
    {
        printf("ERROR STACK OVER FLOW!!!...\n");
    }
    else
    {
        SP = SP + 1;           // Increase SP
        ValOperandST[SP] = ValOperand; // Put data into Stack
    }
}
```

ฟังก์ชัน Push ทำหน้าที่นำข้อมูลเข้าสู่ Stack Push(10) จะได้เป็น Stack = [10] SP = 1

```

float pop() // POP Function
{
    float ValOperand;
    if (SP != 0) // Check Stack NOT EMPTY?
    {
        ValOperand = ValOperandST[SP]; // Get data from Stack
        SP--;                          // Decrease SP
        return (ValOperand);           // Return data
    }
    else
        printf("\nERROR STACK UNDER FLOW!!!...\n");
}

```

ฟังก์ชัน Pop มีหน้าที่ ดึงข้อมูลจาก Stack

ก่อน Pop จะได้เป็น Stack = [10,20] SP = 2

Pop 20

หลัง Pop Stack = [10] SP = 1

```

void CalPostfix(char postfix[80])
{
    float pop1, pop2, value;
    int i, len;
    char ch;
    len = strlen(postfix);
    printf("Postfix = %s\n", postfix);
    for (i = 0; i <= len - 1; i++) // Assign data to OPERAND
    {
        ch = postfix[i]; // Split Character for assign data
        if (strchr("+-* /^", ch) == 0) // Check Is OPERAND?
        {
            printf("\nAssign Number to %c : ", ch);
            scanf("%f", &ValPostfix[i]); // Read data from KBD and direct to Value of OPERAND in Array
        }
    }
    for (i = 0; i <= len - 1; i++) // Calculate Value of POSTFIX
    {
        ch = postfix[i]; // Split Character for prepare to STACK
        if (strchr("+-* /^", ch) == 0) // Check Is OPERAND?
        {
            push(ValPostfix[i]); // push value of OPERAND to Stack
        }
        else
        {
            pop1 = pop(); // Pop 1st
            pop2 = pop(); // pop 2nd

```

```

switch (ch)
{
case '+':
    value = pop2 + pop1; // Calculate
    push(value);         // Push value to Stack
    break;
case '-':
    value = pop2 - pop1;
    push(value);
    break;
case '*':
    value = pop2 * pop1;
    push(value);
    break;
case '/':
    value = pop2 / pop1;
    push(value);
    break;
case '^':
    value = pow(pop2, pop1);
    push(value);
    break;
}
} // End IF
} // End IF
printf("\nANS = %f", pop()); // Last value is ANSWER
} // End Fn.

```

ฟังก์ชัน CalPostfix() เป็นฟังก์ชันหลักสำหรับคำนวณ Postfix

ขั้นตอนที่ 1 แสดง Postfix printf("Postfix = %s\n", postfix);

Postfix = AB+C-d/

ขั้นตอนที่ 2 รับค่าของ Operand for (i = 0; i <= len-1; i++) วนลูปอ่านตัวอักษรทุกตัว

ตรวจสอบว่าเป็น Operand if (strchr("+*/^", ch) == 0) ถ้าไม่ใช่ Operator แสดงว่าเป็น Operand

รับค่าจาก Keyboard scanf("%f",&ValPostfix[i]); ลักษณะการทำงานดังนี้

Assign Number to A : 10

Assign Number to B : 5

Assign Number to C : 3

Assign Number to d : 2

ขั้นตอนที่ 3 คำนวณ Postfix เริ่มวนลูปอีกครั้ง for(i=0;i<=len-1;i++)

กรณีเป็น Operand push(ValPostfix[i]); นำค่าลง Stack

อย่างเช่น อ่าน A = 10 [10] B = 20 [10,20]

กรณีเป็น Operator $\text{pop1} = \text{pop}(); 2\text{pop2} = \text{pop}();$ ดึงข้อมูลออก 2 ค่า

เหตุผลที่ต้อง Pop สองตัว เช่น $10\ 20\ +$ จะได้ผลลัพธ์การคำนวณดังนี้ $10 + 20$

การคำนวณแต่ละ Operator

บวก (+) case '+': $\text{value} = \text{pop2} + \text{pop1};$

ลบ (-) case '-': $\text{value} = \text{pop2} - \text{pop1};$

คูณ (*) case '*': $\text{value} = \text{pop2} * \text{pop1};$

หาร (/) case '/': $\text{value} = \text{pop2} / \text{pop1};$

ยกกำลัง (^) case '^': $\text{value} = \text{pow}(\text{pop2}, \text{pop1});$

ขั้นตอนที่ 4 แสดงผลลัพธ์ $\text{printf}("\text{nANS} = \%f", \text{pop}());$

หลังคำนวณครบแล้ว ค่าที่เหลือใน Stack จะเป็นคำตอบสุดท้าย

```
int main()
{
    printf("POSTFIX CALCULATION PROGRAM\n");
    printf("=====\n");
    CalPostfix(postfix1);
    getch();
    return (0);
} // End Main
```

Main Function เป็นฟังก์ชันหลักของโปรแกรม ลำดับการทำงาน

1.เรียกฟังก์ชันคำนวณ Postfix $\text{CalPostfix}(\text{postfix1});$

2.รับค่าตัวแปรจากผู้ใช้

3.คำนวณผลลัพธ์

4.แสดงคำตอบ

```
PS D:\Bachelor's\ENGCE124\lab4> ./postfix-sum
POSTFIX CALCULATION PROGRAM
=====
Postfix = AB+C-d/

Assign Number to A : 10
Assign Number to B : 5
Assign Number to C : 3
Assign Number to d : 2
ANS = 6.000000
```


Code 4.3 Program Recursion M^N

1. Calculate M^N Function
2. Show each step of recursive result
3. Show final result

```
#include <stdio.h> //use printf()
#include <conio.h> //use getch()
int Number, ans;
```

เรียกใช้ library ที่จะใช้งานในโปรแกรมนี้

```
int Factorial(int N)
{ // Factorial Function
    int x, y;
    if (N == 0)
    {
        printf(".....Roll Back Point\n");
        return (1);
    }
    else
    {
        x = N - 1;
        printf("%2d! = %2d * %2d!\n", N, N, x); // Display before Recursive
        y = Factorial(x);
        printf("%2d! = %2d * %3d = %5d\n", N, N, y, y * N); // Display After Recursive
        return (N * y);
    }
} // End Fn
```

ฟังก์ชัน Factorial() เป็นฟังก์ชันแบบ Recursion หรือฟังก์ชันที่เรียกใช้งานตัวเอง

x ใช้เก็บค่า N-1 y ใช้เก็บค่าที่ส่งกลับมาจากการเรียก Recursion

if (N == 0) {} เป็นเงื่อนไขหยุดการเรียกตัวเอง ตามนิยาม 0! = 1

เมื่อถึงจุดนี้โปรแกรมจะไม่เรียกตัวเองต่อและเริ่มส่งค่ากลับที่ระดับเรียกว่า Roll Back

ตัวอย่าง 5! จะเรียกเป็น 5! 4! 3! 2! 1! 0! เมื่อถึง 0! จะหยุด แล้วส่งค่า 1 กลับขึ้นมา

ในส่วนของ else {} Recursive Case

ขั้นตอนที่ 1 คำนวณ N-1 N = 5 จะได้เป็น x = 4

ขั้นตอนที่ 2 แสดงก่อน Recursive `printf("%2d! = %2d * %2d!\n", N, N, x);`

$$5! = 5 * 4!$$

ขั้นตอนที่ 3 เรียกตัวเอง `y = Factorial(x);`

`Factorial(5)`

`Factorial(4)`

...

`Factorial(0)` เรียกฟังก์ชันตัวเองทำงานวนซ้ำจนถึง 0

ขั้นตอนที่ 4 แสดงหลัง Roll Back `printf("%2d! = %2d * %3d = %5d\n", 2, N, N, y, y*N);`

เมื่อได้รับค่ากลับจาก Recursive แล้ว โปรแกรมจะเริ่มคูณกลับทีละขั้น

$$1! = 1 * 1 = 1$$

$$2! = 2 * 1 = 2$$

$$3! = 3 * 2 = 6$$

$$4! = 4 * 6 = 24$$

$$5! = 5 * 24 = 120$$

ขั้นตอนที่ 5 ส่งค่ากลับ `return(N*y);`

ส่งค่ากลับไปยังระดับก่อนหน้า

```

int main()
{
    printf("RECRSIVE(FACTORIAL) PROGRAM\n");
    printf("=====\n");
    // N=0;
    while (Number != -999)
    {
        printf("Enter Number (-999 is END) : ");
        scanf("%d", &Number);
        if (Number >= 0)
        {
            printf("N! = N(N-1)!\n");
            printf("-----\n");
            ans = Factorial(Number); // Recursive it self
            printf("\nAnswer N! = %d\n", ans);
            printf("-----Finished\n");
            getch();
        }
    } // End while
    return (0);
} // End Main

```

ลำดับการทำงานของโปรแกรม ของ int main()

แสดงชื่อโปรแกรม

รับค่าจากผู้ใช้

เรียกฟังก์ชัน Factorial()

แสดงขั้นตอนการทำงานของ

Recursion

แสดงผลลัพธ์สุดท้าย

หากป้อน -999 โปรแกรมจะจบการทำงาน

```

PS D:\Bachelor's\ENGCE124\lab4> ./factorial
RECRSIVE(FACTORIAL) PROGRAM
=====
Enter Number (-999 is END) : 5
N! = N(N-1)!
-----
5! = 5 * 4!
4! = 4 * 3!
3! = 3 * 2!
2! = 2 * 1!
1! = 1 * 0!
.....Roll Back Point
1! = 1 * 1 = 1
2! = 2 * 1 = 2
3! = 3 * 2 = 6
4! = 4 * 6 = 24
5! = 5 * 24 = 120

Answer N! = 120
-----Finished
Enter Number (-999 is END) : -999
PS D:\Bachelor's\ENGCE124\lab4>

```

ให้นักศึกษานำโค้ดโปรแกรม Recursion ตามตัวอย่างในเอกสาร มาปรับเพิ่มอัลกอริธึมการเรียกตนเอง ของ M^N ใด ๆ พร้อมทั้งอธิบายขบวนการทำงานมาตามลำดับโดยละเอียด

```
#include <stdio.h>
#include <conio.h>

int M,N;
long int ans;

long int Power(int m,int n) {
    long int y;
    if(n==0) {
        printf(".....Roll Back Point\n");
        return 1;
    } else {
        printf("%d^%d = %d * %d^%d\n",m,n,m,m,n-1);
        y = Power(m,n-1);
        printf("%d^%d = %d * %ld = %ld\n",m,n,m,y,m*y);
        return m*y;
    }
}
```

ขั้นที่ 1 Recursive Call

ผู้ใช้ป้อน M,N โปรแกรมเรียกเก็บข้อมูล Power(M,N); เนื่องจาก N ไม่เท่ากับ 0 จึงเรียกตนเองต่อ Power(M,N-1); ทำวนซ้ำไปจนถึง Power(M,0);

ขั้นที่ 2 Base Case

เมื่อทำวนซ้ำถึง Power(M,0); ตรวจสอบเงื่อนไข if(n==0) เป็นจริงโปรแกรมจะสั่งให้ทำงาน โปรแกรมคืนค่า 1 ตามเงื่อนไขโจทย์คือ $M^0 = 1$

ขั้นที่ 3 Roll Back

โปรแกรมย้อนกลับทีละขั้น

ขั้นที่ 1 $2^1 = 2 \times 1 = 2$

ขั้นที่ 2 $2^2 = 2 \times 2 = 4$

ขั้นที่ 3 $2^3 = 2 \times 4 = 8$

ขั้นที่ 4 $2^4 = 2 \times 8 = 16$

ขั้นที่ 5 $2^5 = 2 \times 16 = 32$

```
PS D:\Bachelor's\ENGCE124\lab4> ./recursion
RECURSION POWER PROGRAM
=====
Enter M : 2
Enter N : 5
2^5 = 2 * 2^4
2^4 = 2 * 2^3
2^3 = 2 * 2^2
2^2 = 2 * 2^1
2^1 = 2 * 2^0
.....Roll Back Point
2^1 = 2 * 1 = 2
2^2 = 2 * 2 = 4
2^3 = 2 * 4 = 8
2^4 = 2 * 8 = 16
2^5 = 2 * 16 = 32

Answer 2^5 = 32
PS D:\Bachelor's\ENGCE124\lab4> |
```

สรุปผลการทดลอง

จากการทดลองได้ปรับปรุงโปรแกรม Recursion จากการคำนวณ Factorial ให้สามารถคำนวณค่า M^N ได้ โดยใช้หลักการเรียกฟังก์ชันตนเองตามสมการ $M^N = M \times M^{N-1}$ $M^N = M \times M^{N-1}$ และกำหนดเงื่อนไขหยุดเป็น $M^0 = 1$ โปรแกรมสามารถแสดงลำดับการเรียก Recursion และขั้นตอน Roll Back ได้อย่างชัดเจน ทำให้เข้าใจหลักการทำงานของฟังก์ชันตนเอง การคืนค่ากลับจาก Base Case และกระบวนการคำนวณแบบ Recursive ได้อย่างถูกต้อง ซึ่งเป็นไปตามวัตถุประสงค์ของการทดลอง